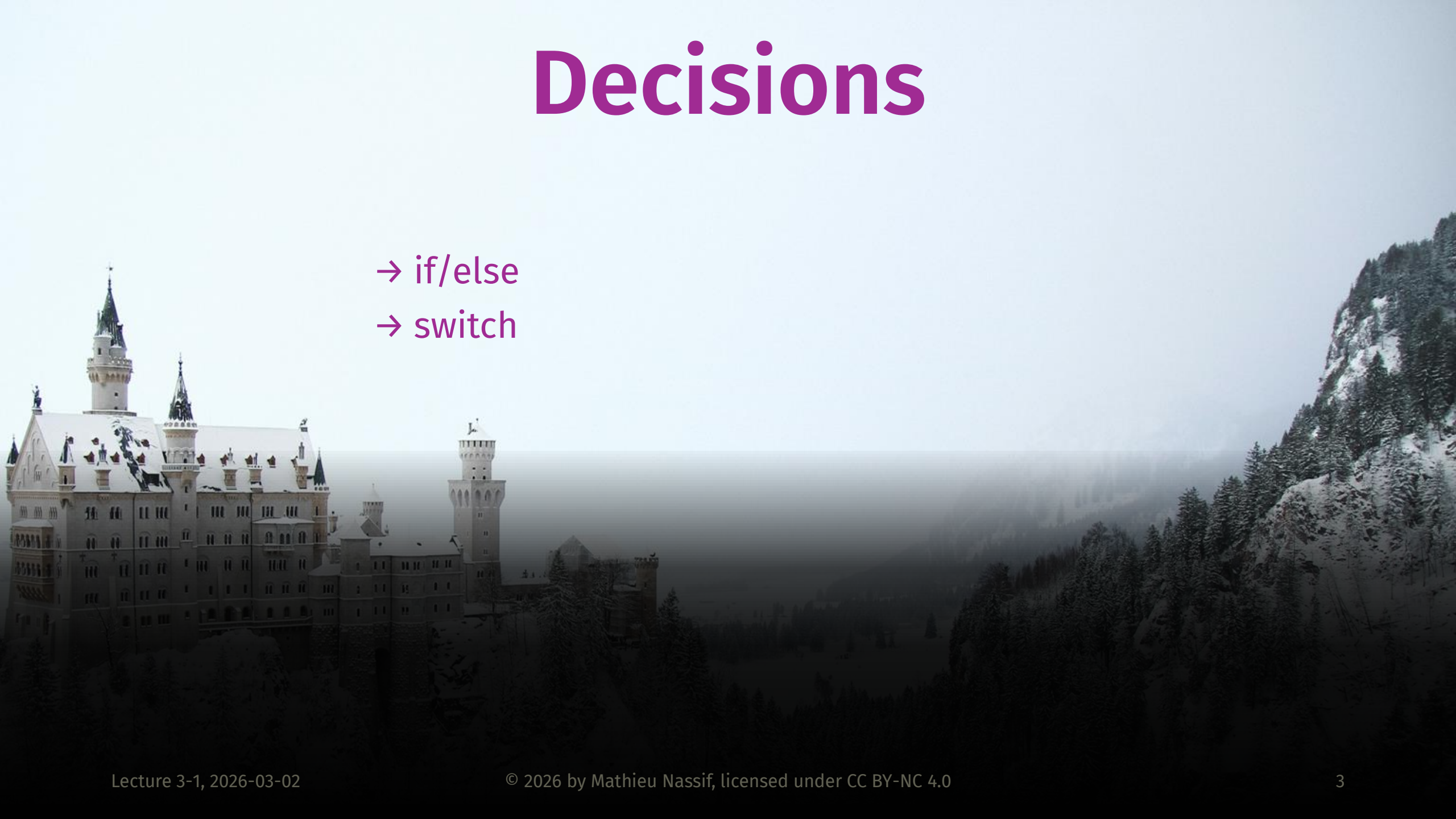
SENG 201
Software Engineering Foundations

[Week 3, Tuesday]



Decisions



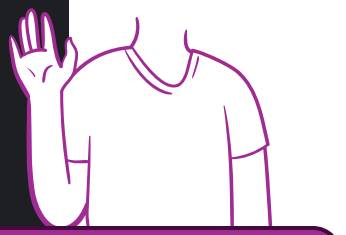
→ if/else

→ switch

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

If, else, else if

```java
private String createCourseString() {
    String courseInfo = "Course " + course.topic() + course.number();
    if (course.topic() == Topic.SENG) {
        courseInfo += " (Yay!)";
    }
    else if (course.topic() == Topic.COSC) {
        courseInfo += " (Ok)";
    }
    else if (course.topic() == Topic.MATH) {
        courseInfo += " (oh no!)";
    }
    else {
        courseInfo += " (what is that!?)";
    }
    return courseInfo;
}
```



What's the problem here?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Switch

switch statement

```
private String createCourseString() {
    String courseInfo = "Course " + course.topic() + course.number();
    switch (course.topic()) {
        case SENG:
            courseInfo += " (Yay!)";
            break;
        case COSC:
        case DATA:
            courseInfo += " (Ok)";
            break;
        case MATH:
            courseInfo += " (oh no!)";
            break;
        default:
            courseInfo += " (what is that!?)";
            break;
    }
    return courseInfo;
}
```

switch expression

```
private String createCourseString() {
    String courseInfo = "Course " + course.topic() + course.number();
    courseInfo += switch (course.topic()) {
        case SENG -> " (Yay!)";
        case COSC, DATA -> " (Ok)";
        case MATH -> " (oh no!)";
        default -> " (IMPOSSIBLE!!)";
    };
    return courseInfo;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Switch

Works only with `byte`, `short`, `char`, `int`, `String`, enums

```java
int x = 1 + 2;
switch (x) {
    case 0:
        System.out.println("zero");
        break;
    case 1:
    case 2:
    case 3:
        System.out.println("small");
        break;
    default:
        System.out.println("large (or negative)");
}
```

```java
String number = "zero";
int x = switch (number) {
    case "zero" -> 0;
    case "one" -> 1;
    case "two" -> 2;
    case "three" -> 3;
    default -> Integer.MAX_VALUE;
};
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

String representation of objects



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Converting an object to a String

to convert any object `x` *to a String, simply call* `x.toString()`

```
List<String> list = List.of("one", "two", "three");
String listAsString = list.toString();
System.out.println(listAsString);
```

```
[one, two, three]
```

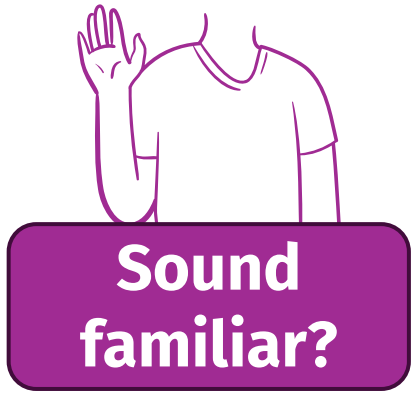
```
Course seng201 = new Course(Topic.SENG, 201);
Notebook notebook = new Notebook(seng201);
notebook.writeNote(new Note("SENG 201 course notes"));
notebook.writeNote(new Note("Topic: software engineering"));

String notebookAsString = notebook.toString();
System.out.println(notebookAsString);
```

```
lecture3_2.Notebook@a22559d2
```

??

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0



Just like with equals()

We need to tell Java how to convert arbitrary objects to String

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Implementing toString() method

```java
@Override
public String toString() {
    return readAllNotes();
}
```

required signature

Must

- Be public
- Return a String
- Be named toString
- Take no argument

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Implementing toString() method

Let’s try with Note:

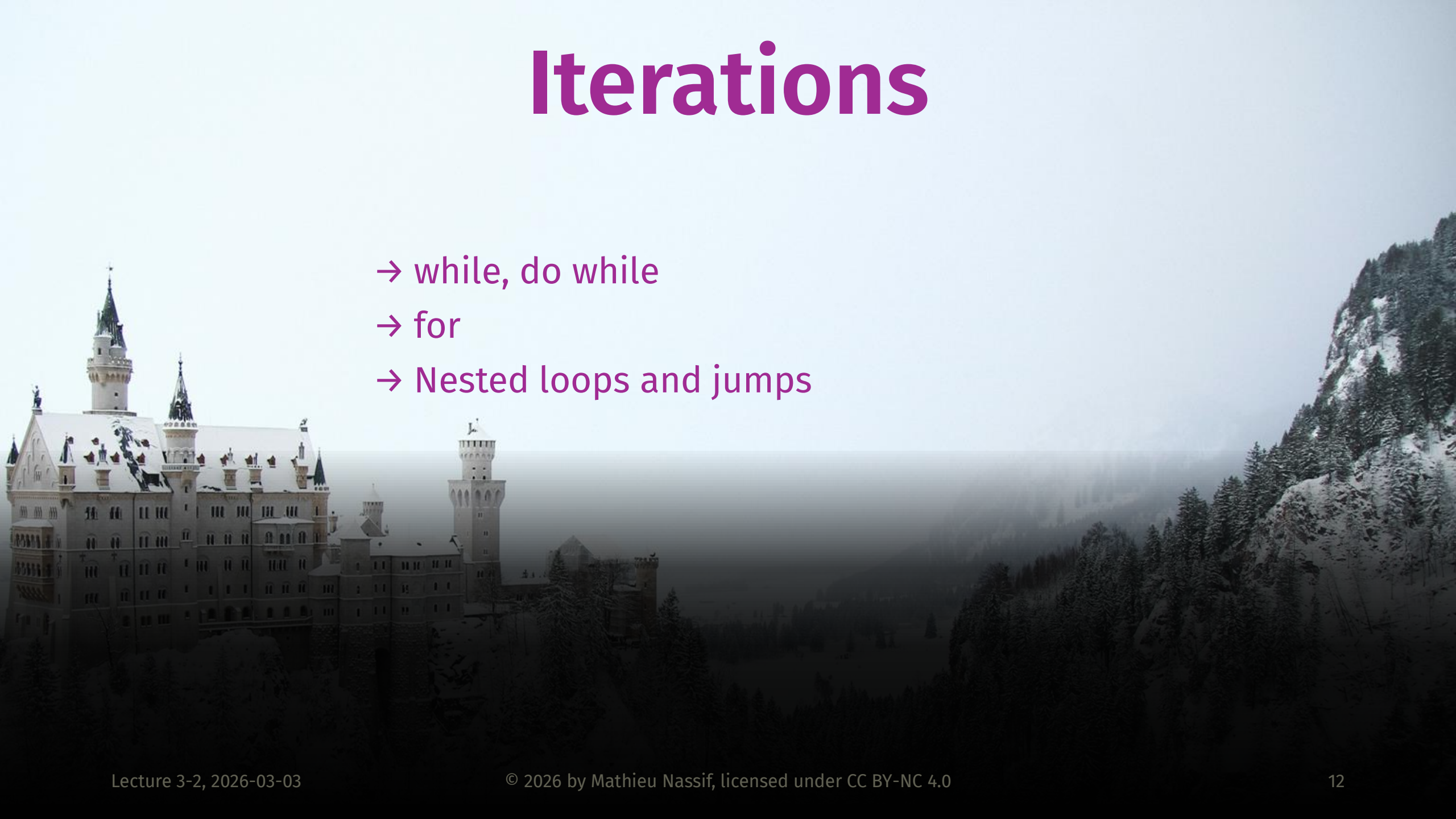
➢ Show content in a frame

```
@Override
public String toString() {
    return "+----------------------\n"
            + content
            + "\n+----------------------";
}
```

Records reimplement toString()

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Iterations



→ while, do while

→ for

→ Nested loops and jumps

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Branching statements in Java

Conditions

- ✓ `if`
- ✓ `if / else if / else`
- ✓ `switch`

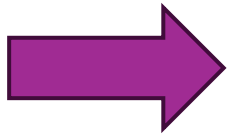
Iterations

- ➢ `for`
- ➢ `for-each`
- ➢ `while`
- ➢ `do-while`

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Let's really *frame* each note

```
+----------------------
Very
important
information
+----------------------
```



```
+-------------+
| Very        |
| important   |
| information |
+-------------+
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Preliminaries: split content by line

```java
@Override
public String toString() {
    String[] lines = content.split("\n");
    int nbLines = lines.length;
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

while / do while / for



- while checks first
- do-while executes at least once
- do-while ends with a ‘;’

While

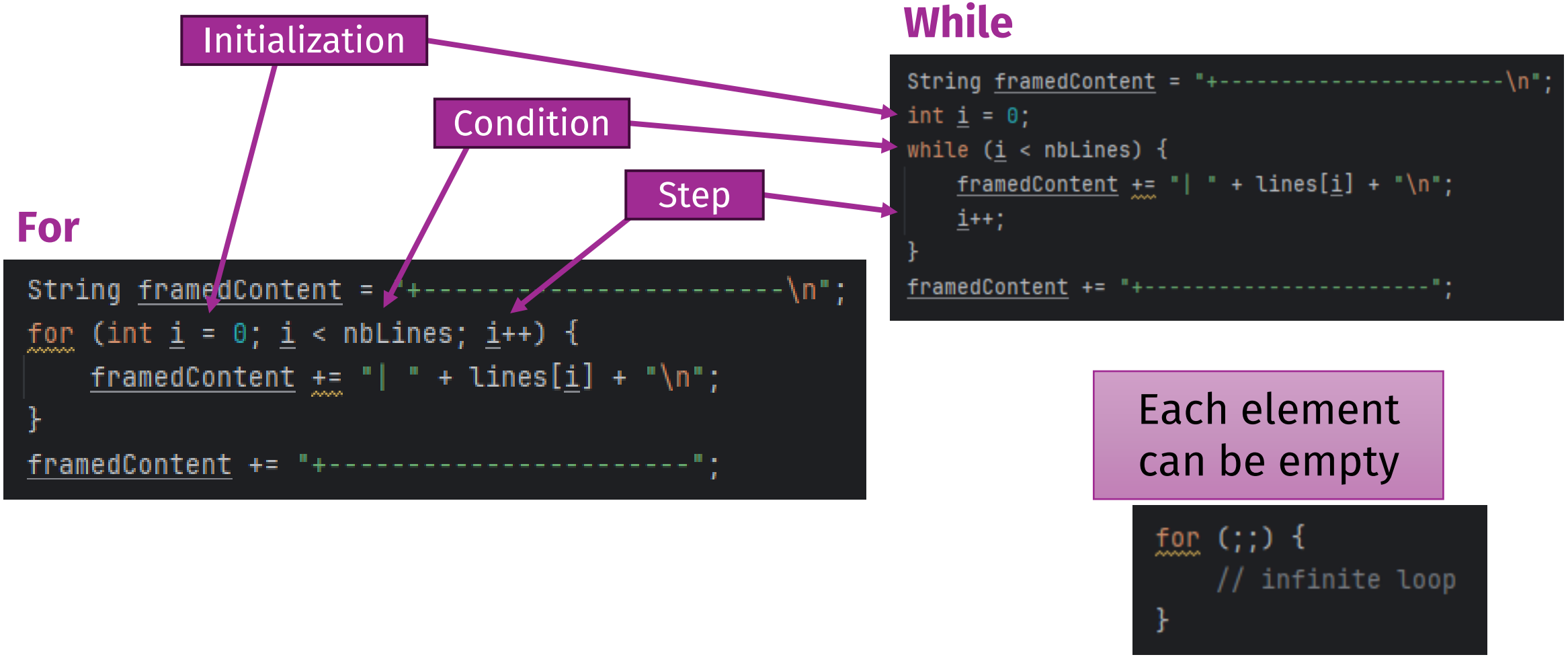
```
String framedContent = "+----------------------\n";
int i = 0;
while (i < nbLines) {
    framedContent += "| " + lines[i] + "\n";
    i++;
}
framedContent += "+----------------------";
```

Do While

```
String framedContent = "+----------------------\n";
int i = 0;
do {
    framedContent += "| " + lines[i] + "\n";
    i++;
} while (i < nbLines);
framedContent += "+----------------------";
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

while / do while / for



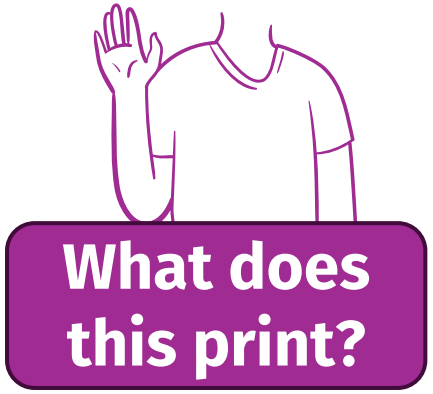
Each element can be empty

```
for (;;) {
    // infinite loop
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

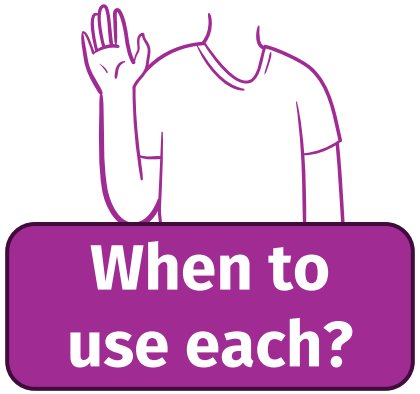
(BAD) Example: for loop

```
int sum = 0;
for (int i = 10; i --> 0; ) {
    sum += i;
}
System.out.println(sum);
```



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

while / do while / for



While

- Unbounded iteration

```
while (!isTaskFinished()) {
    // do more work
}
```

Do While

- Guarantee of one iteration

```
do {
    // start or resume task
} while (!isTaskFinished());
```

For

- Bounded iteration

```
for (int step = 0; step < nbSteps(); step++) {
    // do next step
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

For each

```java
@Override
public String toString() {
    String[] lines = content.split("\n");

    String framedContent = "+----------------------\n";
    for (String line : lines) {
        framedContent += "| " + line + "\n";
    }
    framedContent += "+----------------------";

    return framedContent;
}
```

Iterate over array

- No index for the iteration
- Variable declared in for parentheses

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

For each

In `Notebook.readAllNotes()`

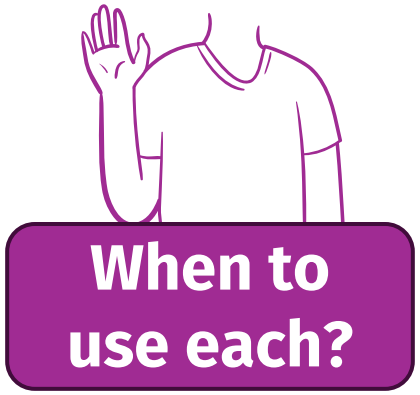
```
fullContent += "\n--------";
for (Note note : notes) {
    fullContent += "\n" + note.content();
}
return fullContent;
```

List<Note> notes

For each works for
- Arrays
- Data structures
- Custom types
- (more tomorrow)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

while / do while / for



While

- Unbounded iteration

```
while (!isTaskFinished()) {
    // do more work
}
```

Do While

- Guarantee of one iteration

```
do {
    // start or resume task
} while (!isTaskFinished());
```

For

- Bounded iteration
- Indexed iteration

```
for (int step = 0; step < nbSteps(); step++) {
    // do next step
}
```

For each

- Data structures
- Unindexed

```
for (Type item : dataStructure) {
    // process item
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Jumping out of iterations

```
String framedContent = "+----------------------\n";
for (String line : lines) {
    if (line.startsWith("//")) {
        // ignore comments
        continue;
    }
    if (line.equals(">> END <<")) {
        break;
    }
    framedContent += "| " + line + "\n";
}
framedContent += "+----------------------";
```

Skip the rest of the current iteration

continue

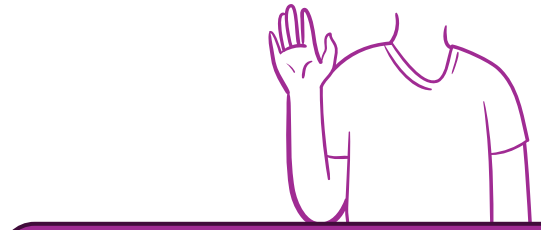
break

Skip the rest of the loop entirely

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Jumping out of iterations

```java
String[] lines = content.split("\n");
for (String line : lines) {
    // regex: split by whitespace
    String[] words = line.split("\\s+");
    for (String word : words) {
        if (word.equals("END_ALL")) {
            break; // end all processing
        }
        if (word.equals("//")) {
            continue; // to next line
        }
        System.out.println(word);
    }
    System.out.println("----");
}
```



How to break out of the outer loop?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

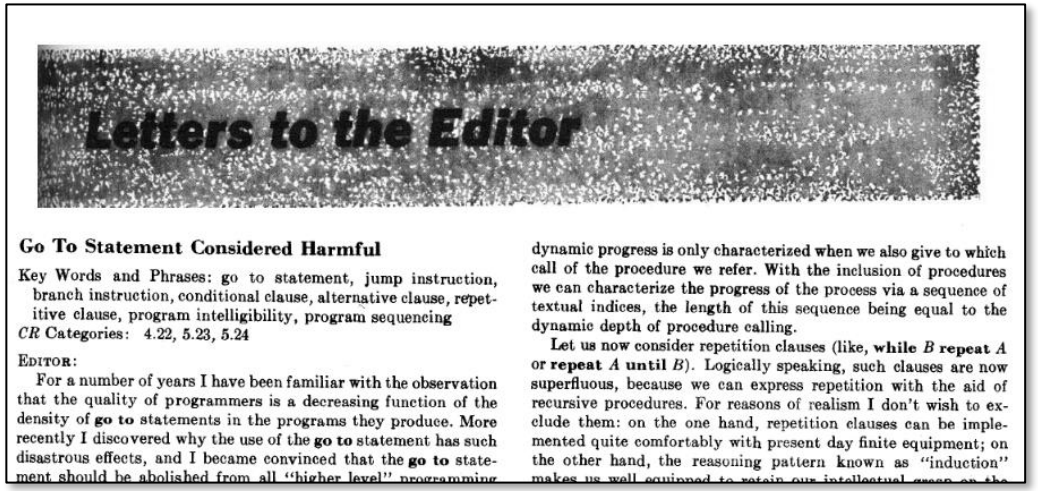
Jumping out of iterations

labels

labels

```java
String[] lines = content.split("\n");
outer: for (String line : lines) {
    // regex: split by whitespace
    String[] words = line.split("\\s+");
    inner: for (String word : words) {
        if (word.equals("END_ALL")) {
            break outer; // end all processing
        }
        if (word.equals("//")) {
            continue outer; // to next line
        }
        System.out.println(word);
    }
    System.out.println("----");
}
```

Don’t use too often!



Letters to the Editor

Go To Statement Considered Harmful

Key Words and Phrases: go to statement, jump instruction, branch instruction, conditional clause, alternative clause, repetitive clause, program intelligibility, program sequencing
CR Categories: 4.22, 5.23, 5.24

EDITOR:

For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of **go to** statements in the programs they produce. More recently I discovered why the use of the **go to** statement has such disastrous effects, and I became convinced that the **go to** statement should be abolished from all "higher level" programming

dynamic progress is only characterized when we also give to which call of the procedure we refer. With the inclusion of procedures we can characterize the progress of the process via a sequence of textual indices, the length of this sequence being equal to the dynamic depth of procedure calling.

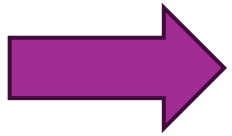
Let us now consider repetition clauses (like, **while** *B* **repeat** *A* or **repeat** *A* **until** *B*). Logically speaking, such clauses are now superfluous, because we can express repetition with the aid of recursive procedures. For reasons of realism I don't wish to exclude them: on the one hand, repetition clauses can be implemented quite comfortably with present day finite equipment; on the other hand, the reasoning pattern known as "induction" makes us well equipped to retain our intellectual grasp on the

Edsger W. Dijkstra, “Go To Statement Considered Harmful.” In Communication of the ACM, 11(3), 1968.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Exercise: Finish the frame (right side)

```
+----------------------
| Very
| important
| information
+----------------------
```



```
+-------------+
| Very        |
| important   |
| information |
+-------------+
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Arrays



- → Constraints
- → Creation
- → Multi-dimensional arrays

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Arrays in Java

- Hold a fixed number of values (can’t be resized)
- All values have the same type
- Zero-indexed (first element at index 0)
  - Access using []

```
someArray[0]
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Declaration, initialization, creation

Declaration

```
String[] strings;
int[] ints;
boolean[] booleans;
```

Creation

```
strings = new String[10];
ints = new int[3];
booleans = new boolean[5];
```

Initialize values to 0/false/null

Declaration + initialization

```
String[] strings = { "a", "b", "c" };
int[] ints = { 1, 2, 3 };
boolean[] booleans = { true, false };
```

Error!

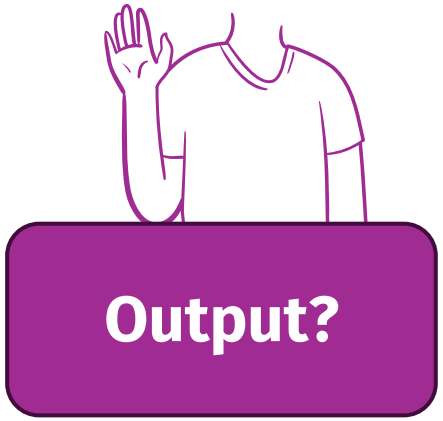
```
strings = { "a", "b", "c" };
ints = { 1, 2, 3 };
booleans = { true, false };
```

Creation + initialization

```
strings = new String[]{ "a", "b", "c" };
ints = new int[]{ 1, 2, 3 };
booleans = new boolean[]{ true, false };
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

```java
String[] original = { "a", "b", "c" };
String[] newValues = original;
newValues[0] = "d";
newValues[1] = "e";
original[0] = "f";
System.out.println(Arrays.toString(newValues));
```



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Multi-dimensional arrays

- An array is declared by `type[]`
- What if `type` is an array?

`String[][]`

- Arrays have a fixed size
- We can change the elements of an array
- What if we reassign internal arrays?

Multi-dimensional arrays are just arrays that contain arrays

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Multi-dimensional arrays

```
String[][] array2d = new String[5][3];
array2d[0][0] = "Hello!";
array2d[2][1] = "middle";
```

```
int[][] points = { {0, 1}, {0, 2}, {1, 1}, {1, 2}};
points[3] = null;
```

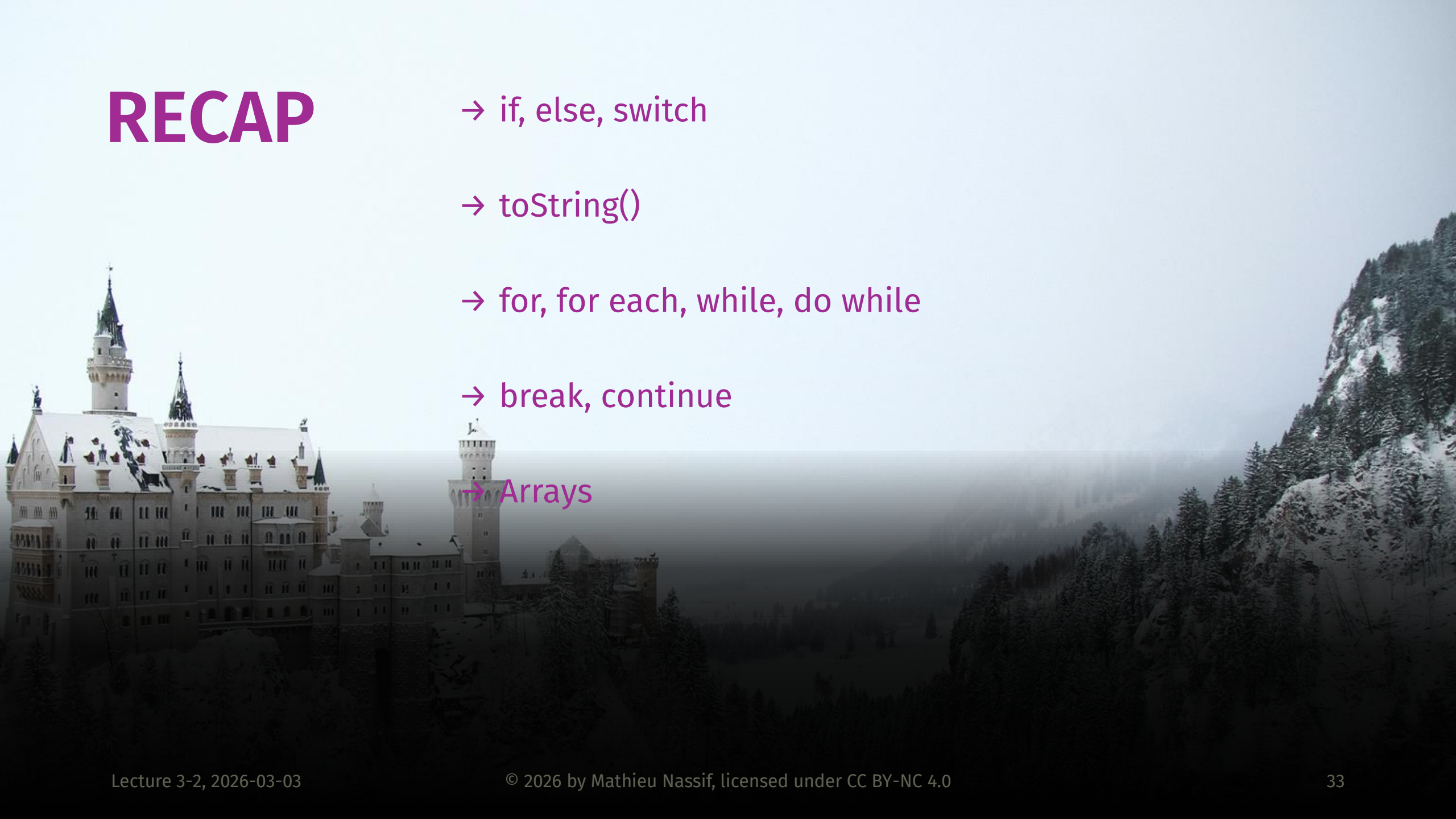
```
array2d[1] = new String[1];
array2d[4] = array2d[1];
array2d[1][0] = "single";
```



```
System.out.println(Arrays.deepToString(array2d));
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

RECAP



→ if, else, switch

→ toString()

→ for, for each, while, do while

→ break, continue

→ Arrays

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0